```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import subprocess, os, time

class CSVApp:
    def __init__(self, root):
        self.root = root
        root.title("CSV Data Processing System")
        root.geometry("1100x700")
        self.df = None
        self.clean_df = None
        self.before = None
        self.dup_before = 0
        self.create_ui()

    def create_ui(self):
        tk.Label(self.root,text="CSV DATA PROCESSING SYSTEM",
                 font=("Arial",22,"bold"),bg="#1f4e78",
                 fg="white",pady=15).pack(fill="x")

        f=tk.Frame(self.root); f.pack(fill="x",padx=20,pady=10)
        self.path=tk.Entry(f,font=("Arial",11))
        self.path.pack(side="left",fill="x",expand=True)

tk.Button(f,text="Browse",width=12,command=self.browse).pack(side="left",pa
dx=5)

        b=tk.Frame(self.root); b.pack(pady=5)
        for text,cmd in [
            ("Load CSV",self.load),("Clean Data",self.clean),
            ("Missing Graph",self.missing_graph),
            ("Analysis Graph",self.analysis_graph),
            ("Export CSV",self.export)]:

tk.Button(b,text=text,width=15,height=2,command=cmd).pack(side="left",padx
=5)

        m=tk.LabelFrame(self.root,text="Pipeline Metrics",font=("Arial",11,"bold"))
```

```
        m.pack(fill="x",padx=20,pady=10)

        self.labels={}
        names=["Rows Loaded","Columns","Missing Before","Missing After",
               "Duplicates Before","Duplicates After","Rows After
Cleaning","Processing Time"]
        for i,n in enumerate(names):
            x=tk.Frame(m,padx=15,pady=5); x.grid(row=i//4,column=i%4)
            tk.Label(x,text=n,font=("Arial",9,"bold")).pack()
            self.labels[n]=tk.Label(x,text="-",font=("Arial",12))
            self.labels[n].pack()

        tf=tk.LabelFrame(self.root,text="Dataset Preview",
                         font=("Arial",11,"bold"))
        tf.pack(fill="both",expand=True,padx=20,pady=5)

        self.tree=ttk.Treeview(tf,show="headings")
        sy=ttk.Scrollbar(tf,orient="vertical",command=self.tree.yview)
        sx=ttk.Scrollbar(tf,orient="horizontal",command=self.tree.xview)
        self.tree.configure(yscrollcommand=sy.set,xscrollcommand=sx.set)
        self.tree.pack(side="left",fill="both",expand=True)
        sy.pack(side="right",fill="y")
        sx.pack(side="bottom",fill="x")

        self.status=tk.Label(self.root,text="Ready. Click Browse to select a CSV.",
                             anchor="w")
        self.status.pack(fill="x",padx=20,pady=5)

    def browse(self):
        try:
            r=subprocess.run(
                ["powershell.exe","-NoProfile","-Command",
                 'Add-Type -AssemblyName System.Windows.Forms; '
                 '$d=New-Object System.Windows.Forms.OpenFileDialog; '
                 '$d.Title="Select CSV File"; '
                 '$d.Filter="CSV Files (*.csv)|*.csv|All Files (*.*)|*.*"; '
                 'if($d.ShowDialog() -eq "OK"){Write-Output $d.FileName}'],
                capture_output=True,text=True)
            p=r.stdout.strip()
            if p:
                p=subprocess.run(["wslpath","-u",p],
```

```
                        capture_output=True,text=True).stdout.strip()
            self.path.delete(0,tk.END)
            self.path.insert(0,p)
            self.status.config(text="CSV selected. Click Load CSV.")
    except Exception as e:
        messagebox.showerror("Error",str(e))

def load(self):
    try:
        p=self.path.get().strip()
        if not p: raise Exception("Please select a CSV file.")
        self.df=pd.read_csv(p)
        self.clean_df=None
        self.before=self.df.isnull().sum()
        self.dup_before=int(self.df.duplicated().sum())
        self.show(self.df)
        self.metrics()
        self.status.config(text="CSV loaded successfully.")
    except Exception as e:
        messagebox.showerror("Load Error",str(e))

def clean(self):
    try:
        if self.df is None: raise Exception("Load a CSV file first.")
        t=time.time()
        d=self.df.drop_duplicates().copy()

        for c in d.select_dtypes(include=np.number).columns:
            d[c]=d[c].fillna(d[c].mean())

        for c in d.select_dtypes(include=["object","string"]).columns:
            d[c]=d[c].fillna("Unknown").astype(str).str.strip()

        self.clean_df=d
        self.show(d)
        self.metrics(time.time()-t)
        self.status.config(text="Data cleaned successfully.")
    except Exception as e:
        messagebox.showerror("Cleaning Error",str(e))

def show(self,d):
```

```
        self.tree.delete(*self.tree.get_children())
        self.tree["columns"]=list(d.columns)

        for c in d.columns:
            self.tree.heading(c,text=c)
            self.tree.column(c,width=140,anchor="center")

        for _,r in d.iterrows():
            vals=[]
            for v in r:
           if pd.isna(v): vals.append("Missing")
                elif isinstance(v,(float,np.floating)): vals.append(f"{v:.2f}")
                else: vals.append(str(v))
            self.tree.insert("", "end", values=vals)

    def metrics(self,t=0):
        if self.df is not None:
            self.labels["Rows Loaded"].config(text=len(self.df))
            self.labels["Columns"].config(text=len(self.df.columns))
            self.labels["Missing Before"].config(text=int(self.before.sum()))
            self.labels["Duplicates Before"].config(text=self.dup_before)

        if self.clean_df is not None:
            self.labels["Missing After"].config(
                text=int(self.clean_df.isnull().sum().sum()))
            self.labels["Duplicates After"].config(
                text=int(self.clean_df.duplicated().sum()))
            self.labels["Rows After Cleaning"].config(
                text=len(self.clean_df))
            self.labels["Processing Time"].config(text=f"{t:.4f} sec")

    def missing_graph(self):
    try:
            if self.df is None or self.clean_df is None:
                raise Exception("Load and clean the data first.")
            a=self.before
            b=self.clean_df.isnull().sum()
            x=np.arange(len(a))
            plt.figure(figsize=(10,5))
            plt.bar(x-.2,a,.4,label="Before Cleaning")
            plt.bar(x+.2,b,.4,label="After Cleaning")
```

```
            plt.xticks(x,a.index,rotation=45)
            plt.ylabel("Missing Values")
            plt.title("Missing Values Before and After Cleaning")
            plt.legend(); plt.tight_layout()
            plt.savefig("missing_values_comparison.png")
        plt.show()
    except Exception as e:
            messagebox.showerror("Graph Error",str(e))

    def analysis_graph(self):
        try:
            if self.clean_df is None:
                raise Exception("Clean the data first.")
            nums=list(self.clean_df.select_dtypes(include=np.number).columns)

cats=list(self.clean_df.select_dtypes(include=["object","string"]).columns)
            if not nums or not cats:
                raise Exception("CSV needs one text and one numeric column.")
            d=self.clean_df.groupby(cats[0])[nums[0]].mean()
            plt.figure(figsize=(9,5))
            d.plot(kind="bar")
            plt.title(f"Average {nums[0]} by {cats[0]}")
            plt.ylabel(f"Average {nums[0]}")
            plt.xticks(rotation=45)
            plt.tight_layout()
        plt.savefig("data_analysis_chart.png")
        plt.show()
    except Exception as e:
        messagebox.showerror("Graph Error",str(e))

    def export(self):
        try:
            if self.clean_df is None:
            raise Exception("Clean the data first.")
            p=filedialog.asksaveasfilename(
                title="Save Cleaned CSV",
                defaultextension=".csv",
                filetypes=[("CSV Files","*.csv")])
            if p:
                self.clean_df.to_csv(p,index=False)
                self.status.config(text="Cleaned CSV exported successfully.")
```

```
            messagebox.showinfo("Success","Cleaned CSV saved successfully.")
        except Exception as e:
            messagebox.showerror("Export Error",str(e))

root=tk.Tk()
CSVApp(root)
root.mainloop()
```